

PROYECTO 1

Procesamiento y Análisis de Datos Masivos

Prof. Heider Sanchez

OBJETIVO

El presente proyecto tiene como objetivo aplicar las técnicas estudiadas en clase para procesar, analizar y extraer conocimiento a partir de grandes volúmenes de datos, utilizando el MovieLens 20M Dataset como caso de estudio. El dataset está disponible en: <https://grouplens.org/datasets/movielens/20m/>

Al concluir el proyecto, el estudiante habrá:

- Dominado herramientas de procesamiento distribuido (Apache Spark / PySpark) para el procesamiento a escala de datos masivos.
- Implementado técnicas de similitud, MinHashing y LSH, tanto de forma manual como mediante Spark MLlib, para identificar películas similares.
- Descubierta reglas de asociación que revelen patrones de comportamiento de usuarios.
- Analizado la escalabilidad, eficiencia y aplicabilidad de las técnicas en sistemas de recomendación reales.

CONTEXTO DEL DATASET

MovieLens 20M contiene 20 millones de calificaciones de más de 138,000 usuarios a casi 27,000 películas, además de tags asignados por los propios usuarios. Representa un escenario clásico de sistemas de recomendación y minería de patrones en datos masivos, con suficiente volumen para evidenciar los retos de escalabilidad inherentes a cada técnica estudiada. El tamaño del dataset hace de Apache Spark una herramienta natural para su procesamiento: PySpark permite distribuir el cálculo de conteos, rankings, firmas MinHash y reglas de asociación, mientras que Spark MLlib ofrece implementaciones nativas de MinHashLSH dentro de su módulo de features.

PARTE I. PREPROCESAMIENTO Y EXPLORACIÓN DE DATOS (EDA)

Todas las tareas de esta parte pueden realizarse con PySpark utilizando Spark SQL y la API de DataFrames, lo que permite operar sobre el dataset completo sin necesidad de cargarlo en memoria local.

1.1 Preprocesamiento

- Eliminar registros con valores nulos, inconsistentes o duplicados.
- Unificar etiquetas de géneros (e.g., “Sci-Fi” → “Science Fiction”).
- Transformar las calificaciones a escala binaria (like / dislike) con un umbral definido y justificado por el equipo (e.g., calificación ≥ 3.5 = like).

1.2 Exploración de Datos (EDA)

Realizar un análisis exploratorio que permita comprender la estructura y distribución del dataset antes de aplicar las técnicas de procesamiento. Los hallazgos deben presentarse en el reporte con los gráficos que los sustenten. Se esperan, al menos, los siguientes análisis:

- Distribución de calificaciones (histograma de ratings).
- Número de calificaciones por usuario (distribución, media, mediana, usuarios con mayor actividad).
- Número de calificaciones por película (distribución, películas más y menos calificadas).

- Distribución de géneros en el catálogo y en las calificaciones.
- Análisis de la densidad de la matriz usuario-película (sparsity).
- Evolución temporal de las calificaciones (si el dataset incluye timestamps).
- Cualquier hallazgo adicional que el equipo considere relevante para motivar las decisiones de modelado.

Cada hallazgo debe estar acompañado del gráfico correspondiente (histograma, boxplot, heatmap, serie de tiempo, etc.) y de una breve interpretación escrita.

PARTE II. PROCESAMIENTO DISTRIBUIDO CON SPARK

Esta parte debe implementarse con PySpark. Se pueden utilizar tanto la API de RDDs (estilo MapReduce) como la API de DataFrames / Spark SQL; el equipo debe justificar la elección según el tipo de operación.

- Contar el número de calificaciones por película y por usuario mediante operaciones distribuidas.
- Calcular un ranking de popularidad de películas (top 20).
- Comparar, al menos para una operación representativa, el tiempo de ejecución con RDDs vs. DataFrames/Spark SQL.
- Presentar los resultados en tablas y gráficas de barras ordenadas por popularidad.

PARTE III. SIMILITUD, MINHASHING Y LSH

Dada la escala del dataset, Apache Spark es una opción viable para todas las tareas de esta parte. El equipo puede optar por: (a) implementar MinHash y LSH manualmente sobre RDDs/DataFrames; (b) utilizar la implementación nativa de MLlib (*MinHashLSH*); o (c) combinar ambos enfoques para comparar resultados. Cualquiera que sea la elección, debe justificarse en el reporte e indicar explícitamente qué tareas se resolvieron con Spark y cuáles, en su caso, se ejecutaron en local con una muestra del dataset.

3.1 Estrategia de representación

El equipo debe elegir y justificar una de las siguientes representaciones para modelar las películas como conjuntos:

- Opción A - Usuarios: cada película se representa como el conjunto de usuarios que la calificaron positivamente.
- Opción B - Tags: cada película se representa como el conjunto de tags asignados por los usuarios.
- Opción C - Combinada: explorar ambas representaciones y comparar resultados.

La justificación debe argumentar por qué la representación elegida es adecuada para el objetivo de identificar películas similares.

3.2 Similitud de Jaccard exacta vs. aproximada

- Implementar el cálculo exacto de la similitud de Jaccard entre pares de películas.
- Implementar MinHashing para obtener una estimación eficiente de Jaccard.
- Comparar ambos métodos mediante métricas cuantitativas (e.g., error absoluto medio, correlación de Pearson) sobre una muestra representativa de pares.
- Presentar gráficas que ilustren la relación entre el número de funciones hash y la precisión de la aproximación.

3.3 Locality Sensitive Hashing (LSH)

Implementar LSH sobre las firmas MinHash para reducir el espacio de pares candidatos. Se requiere:

- Variar el número de bandas b y el número de filas por banda r , manteniendo $b \times r$ igual al número total de funciones hash.
- Para cada configuración, medir y reportar:
 - Falsos positivos: pares detectados como similares por LSH cuya similitud de Jaccard exacta es baja.
 - Falsos negativos: pares con alta similitud de Jaccard que LSH no recuperó como candidatos.
- Generar curvas de precisión y recall en función del número de bandas, evidenciando el trade-off entre eficiencia computacional y calidad de los resultados.
- Discutir cómo la elección de b y r determina el umbral de similitud efectivo de LSH y el volumen de candidatos generados.

PARTE IV. REGLAS DE ASOCIACIÓN Y CONJUNTOS FRECUENTES

Spark MLlib incluye una implementación distribuida de FP-Growth que escala bien sobre el dataset completo. El uso de Apriori en local sobre una muestra es también aceptable si el equipo argumenta la justificación de tamaño de muestra.

- Aplicar el algoritmo Apriori (o FP-Growth) para descubrir reglas de asociación entre películas con calificación positiva.
- Ejemplo ilustrativo: “Usuarios que calificaron positivamente Matrix también calificaron positivamente Terminator 2”.
- Evaluar cada regla con métricas de soporte, confianza y lift.
- Reportar un mínimo de diez reglas relevantes con sus métricas.

PARTE V. DISCUSIÓN Y REFLEXIÓN

- Comparar exactitud frente a eficiencia: ¿cuánto se pierde en precisión al usar MinHash y LSH respecto al cálculo exacto de Jaccard?
- Analizar el impacto de los falsos positivos y falsos negativos de LSH en un sistema de recomendación real.
- Evaluar qué tareas del proyecto se beneficiaron de la ejecución distribuida con Spark y cuáles hubieran sido manejables en memoria local: ¿valía la pena el overhead de Spark en cada caso?
- Identificar cuellos de botella de escalabilidad: uso de memoria, tiempo de ejecución y tamaño de matrices.
- Reflexionar sobre la aplicabilidad de estas técnicas en plataformas como Netflix, Spotify o Amazon.

ENTREGABLES ESPERADOS

1. Código fuente

- Implementaciones en Python con PySpark.
- Scripts organizados por módulos: preprocesamiento/EDA, procesamiento distribuido, MinHashing/LSH y reglas de asociación.
- Cada módulo debe indicar claramente si la tarea se ejecuta en Spark (distribuido) o en local, y por qué.
- Archivo README con instrucciones de ejecución, dependencias y ejemplos de uso.

2. Reporte técnico (máximo 8 páginas)

El reporte debe ser conciso y privilegiar la claridad sobre la extensión. Se organiza en las siguientes secciones:

1. Metodología: descripción de los pasos aplicados en cada parte del proyecto.
2. Hallazgos EDA: presentar los principales resultados del análisis exploratorio. Cada hallazgo debe estar respaldado por al menos un gráfico (histograma, boxplot, heatmap, serie de tiempo, etc.) y una interpretación escrita breve que explique su relevancia para el proyecto.
3. Resultados: tablas, métricas y gráficas de las Partes II, III y IV (top 20 películas, curvas precisión/recall de LSH, reglas de asociación, etc.).
4. Discusión crítica: respuestas a los puntos planteados en la Parte V.

Nota: los gráficos de EDA y las curvas de desempeño vs. calidad de LSH (precisión/recall por configuración de bandas) son requisitos indispensables para obtener el puntaje completo en los criterios de Análisis y resultados.

RÚBRICA DE EVALUACIÓN (20 PUNTOS)

Criterio	Descripción	Pts.
Preprocesamiento de datos	Limpieza, transformación binaria y unificación de géneros.	5
Implementación técnica	MapReduce/Spark, MinHashing, LSH y reglas de asociación.	6
Análisis y resultados	Claridad de visualizaciones, gráficos EDA y curvas de desempeño LSH.	4
Discusión crítica	Reflexión sobre escalabilidad, falsos positivos/negativos y aplicabilidad.	3
Código y reproducibilidad	Código comentado, ejecutable y con instrucciones en README.	2
Total		20